CPSC 250
Lists, Tuples, and Sets

Edward Brash

1 Lists

Python lists are complex data structures that store a sequence of information. Lists usually contain related information, but they can contain multiple data types, including other lists.

```
grades = [52, 78, 100, 92, 32]
```

Each element in the list has an index.

Index	Value
0	52
1	78
2	100
3	92
4	32

1.1 Iterating Through a List

```
for i in range(len(grades)):
    print(i, grades[i], id(grades[i]))
```

This prints the index, the value stored at that index, and the memory identity of the object.

1.2 A Conceptual Memory Picture

It is useful to imagine a list as a sequence of memory locations:

Index	Value	Conceptual Memory Location
0	52	31
1	78	32
2	100	33
3	92	34
4	32	35

However, this picture is only conceptual. Python does a lot of things under the hood to optimize list storage. The actual storage should not necessarily be interpreted as a simple sequential block of values.

1.3 Lists Can Store Mixed Information

```
employee_info = [
    "John",
    "Smith",
    1289,
    "Promenade Lane",
    229724182
]
```

This list contains a first name, last name, street number, street name, and Social Security number. Python permits this kind of mixed storage, although in larger programs we should be careful about whether this is the cleanest design.

2 Python List Methods

Some common list operations include:

- `append(value)`: add a value to the end of the list.
- `insert(index, value)`: insert a value at a particular position.
- `remove(value)`: remove the first occurrence of a value.
- `pop()`: remove and return the last element.
- `sort()`: sort the list in place.
- `reverse()`: reverse the list in place.

```
nums = [3, 1, 4]
nums.append(10)
nums.sort()
print(nums)
```

3 Tuples

Tuples are sequence-like structures that cannot be changed once created. We say that tuples are **immutable**.

List	`[1, 2, 3]`
Tuple	`(1, 2, 3)`

Examples:

```
my_list = [1, 2, 3]
my_tuple = (1, 2, 3)
```

Tuples are useful for data that should not change. They are often used for constants or for returning multiple values from a function.

Honestly, tuples are not always the first structure students reach for, but their immutability can be very useful.

4 Named Tuples

One problem with lists and ordinary tuples is that referring to elements by index can make code hard to read.

```
car = ('Mercedes', 'E350', 62000)
print(car[2])
```

The expression `car[2]` works, but it is not especially descriptive. A reader has to remember that index 2 means the price.

Python's `collections` package provides `namedtuple`, which allows us to refer to tuple elements by name.

```
from collections import namedtuple

Car = namedtuple('Car', ['make', 'model', 'price'])

mercedes = Car('Mercedes', 'E350', 62000)
bmw = Car('BMW', '325i', 67000)

print(mercedes.price)
print(bmw.model)
```

This produces code that is easier to read and less dependent on remembering numerical positions.

5 Sets

Sets in Python are unordered collections of unique objects. They behave much like sets in mathematics.

```
my_set = {1, 2, 3, 17}
```

Sets use curly braces:

$$\{\}$$

5.1 List, Tuple, and Set Syntax

Structure	Syntax	Mutable?
List	[ ]	Yes
Tuple	( )	No
Set	{ }	Yes

6 Using Sets to Remove Duplicates

Sets automatically eliminate duplicate values.

```
first_names = [
    'bob',
    'alice',
    'jane',
    'bob',
    'fred',
```

```
    'alice'
]

first_names_set = set(first_names)

print(first_names_set)
```

Possible output:

```
{'bob', 'alice', 'jane', 'fred'}
```

The order of the output is not guaranteed, because sets are unordered.

7 Set Methods

Useful set methods include:

- `len(my_set)` — return the number of elements.
- `set1.update(set2)` — add all elements from another set.
- `my_set.add(value)` — add one value.
- `my_set.remove(value)` — remove a value.
- `my_set.pop()` — remove and return an arbitrary value.
- `my_set.clear()` — remove all values.

7.1 Important Warning About `pop()`

For lists, `pop()` removes the last element by default. For sets, however, there is no "last" element because sets are unordered.

Therefore:

```
my_set.pop()
```

removes an arbitrary element. You should not write code that depends on which element will be removed.

8 Mathematical Set Operations

Python sets support standard mathematical operations.

8.1 Union

The union of two sets contains everything that appears in either set.

$$A \cup B$$

```
A.union(B)
```

8.2 Intersection

The intersection contains only elements that appear in both sets.

$$A \cap B$$

```
A.intersection(B)
```

8.3 Difference

The difference contains elements in one set but not the other.

$$A - B$$

```
A.difference(B)
```

8.4 Symmetric Difference

The symmetric difference contains elements that appear in exactly one of the two sets.

```
A.symmetric_difference(B)
```

9 Key Takeaways

- Lists are ordered, mutable collections.
- Lists can hold mixed data types, but that flexibility should be used carefully.
- Tuples are ordered but immutable.
- Named tuples make tuple-based code easier to read.
- Sets are unordered collections of unique values.
- Sets are useful for removing duplicates and performing mathematical set operations.